Mod optimization

From PZwiki

English • čeština • Deutsch • español • français • italiano • 日本語 • 한국어 • polski • português • português do Brasil • русский • ไทย • Türkçe • українська • Tiếng Việt • 中文（简体） • 中文（繁體）

Modding

Give your feedback on the Modding Wiki **here (https://docs.google.com/forms/d/e/1FAIpQLSdxBWwKUcNP_KbzmmWo1eNc2QkpWk5dTj4PMhuNLY9_dc4IEw/viewform?usp=header)**!

Category:Modding • Scripts • Lua (API) • Java • User Interface • Modeling • Animation • Texturing • Mapping • Rendering • Translation

This page has been updated to an *unstable* beta version (42.10.0).

Mod optimization is a very important part of mod making which is often ignored by too many people. When mods start to stack in a save, you will happen to drag all the problems of each mods and as such can create important performance impact when very simple tips and tricks can be utilized to improve performances. The following guide will explain in detail some of these tips and tricks to reduce the performance impact of your mod to a minimum and develop good habits.

Lua (API)

Don't need it? Don't run it

The first and main way you can and should optimize your code is with a simple rule: if you don't need to run it, don't do it. It might seem obvious at first and yet it is often a common mistake.

In your code, you will most likely use checks to verify if a code should run, for example run your code only if the player moves, as an intended feature. However it's possible to use the same principle to stop code from running when it doesn't need to run even if it technically doesn't matter in term of gameplay for your mod. It will however matter internally to have less code running.

Examples

Optimizing code that runs a lot, like for zombies for example, matters especially a lot. Let's take for example a simple mod with the goal of setting every zombies as skeletons using `IsoZombie.setSkeleton`. **Source:** `ProjectZomboid\zombie\characters\IsoZombie.class`

Retrieved: *Build 42.0.1*

```
public void setSkeleton(boolean var1) {
   this.isSkeleton = var1;
   if (var1) {
      this.getHumanVisual().setHairModel("");
      this.getHumanVisual().setBeardModel("");
      ModelManager.instance.Reset(this);
   }
}
```

```
-- Iterate through every zombies and set their visual as skeletons
local function OnZombieUpdate(zombie)
    zombie:setSkeleton(true)
end

Events.OnZombieUpdate.Add(OnZombieUpdate)
```

This code will run for every zombies, every ticks which can become extremely costly because the `setSkeleton` function calls a lot of stuff, thus running a lot more code than we really need because once the zombie is made as a skeleton, it doesn't need to be set again. To greatly reduce the performance impact of this

code, we can add a simple check and ignore the zombie if it's already a skeleton:

```lua
-- Iterate through every zombies and set their visual as skeletons only if they aren't skeletons
local function OnZombieUpdate(zombie)
    if not zombie:isSkeleton() then
        zombie:setSkeleton(true)
    end
end

Events.OnZombieUpdate.Add(OnZombieUpdate)
```

This equals to a single function call per zombie every ticks, and ignores multiple function calls if the zombie is already a skeleton, greatly reducing the amount of code ran per ticks.

If you want to go further in the optimization, there are cases where doing such a check is possible and yet is actually worse for performances. Take the example of keeping the hit time of a zombie to a fixed value. **Source:** `ProjectZomboid\zombie\characters\IsoZombie.class`

Retrieved: *Build 42.0.1*

```java
public int getHitTime() {
    return this.hitTime;
}

public void setHitTime(int var1) {
    this.hitTime = var1;
}
```

If we apply the trick presented before, we obtain the code below:

```lua
-- Iterate through every zombies and set their hit time to the desired value
local desiredHitTime = 3
local function OnZombieUpdate(zombie)
```

```
    -- verify current zombie hit time is the desired value
    if zombie:getHitTime() != desiredHitTime then
        -- set the value
        zombie:setHitTime(desiredHitTime)
    end
end

Events.OnZombieUpdate.Add(OnZombieUpdate)
```

The code ran will do a single java function call and comparison operation. But we do one less code operation with the code below:

```
-- Iterate through every zombies and set their hit time to the desired value
local desiredHitTime = 3
local function OnZombieUpdate(zombie)
    -- set the value
    zombie:setHitTime(desiredHitTime)
end

Events.OnZombieUpdate.Add(OnZombieUpdate)
```

In this last example, we have a single java function call and nothing more. These exceptions require the decompilation of the game code to know what you are using and optimize your code. Doing checks anyway is however usually fine to do.

Local, not global

Accessing a global is more costly than having a variable be local and poses other problems (see Local and global variables). This can simply be achieved by never using global variables. This principle is the direct reason why caching should be used, as described in a later part.

```
-- bad
myVariable = "Hello World!"
```

```lua
-- better
local myVariable = "Hello World!"
```

```lua
-- bad
MyFunctions = {}

MyFunctions.SayHelloWorld = function()
    print("Hello World!")
end

-- better
local MyFunctions = {}

MyFunctions.SayHelloWorld = function()
    print("Hello World!")
end
```

There are however cases of vanilla global variables which you need to access, or sometimes you really need to have your variable global. In these rare cases, you can directly cache the global variable into a local variable:

```lua
-- File1.lua
MyGlobalVariable = "Hello World!"

-- File2.lua
local MyGlobalVariable = MyGlobalVariable -- store locally the global variable
```

Source: `ProjectZomboid\media\lua\server\Items\ISDynamicRadio.lua`

Retrieved: *Build 42.0.2*

```lua
ProceduralDistributions = {};
```

```
-- Making a vanilla global as local
local ProceduralDistributions = ProceduralDistributions
```

Caching

The best way to optimize code is to not run the code. This can be applied with a very easy method of optimizing, and also cleaning up your code at the same time, which is called caching. Caching involves storing a value or object to be used later instead of retrieving it every time we need it.

Simple caching within a function

Below is a small code snipet which will access the item in the hands of the player, and if it is a gun it changes the ammo count to the maximum.

```
local function OnPlayerUpdate(player)
    -- check if player has a weapon
    if player:getPrimaryHandItem() and instanceof(player:getPrimaryHandItem(),"HandWeapon") then
        -- verify it's a gun
        if player:getPrimaryHandItem():isRanged() then
            -- verify it has the max amount of bullets
            if player:getPrimaryHandItem():getCurrentAmmoCount() ~= player:getPrimaryHandItem():getMaxAmmo() then
                -- set the bullet count to max
                player:getPrimaryHandItem():setCurrentAmmoCount(player:getPrimaryHandItem():getMaxAmmo())
            end
        end
    end
end

Events.OnPlayerUpdate.Add(OnPlayerUpdate)
```

We can make this code easier to read, organize it better as well as optimize it by using caching.

```lua
local function OnPlayerUpdate(player)
    -- check if player has a gun
    local weapon = player:getPrimaryHandItem() -- caching the item in primary hand
    if weapon and instanceof(weapon,"HandWeapon") and weapon:isRanged() then
        -- verify it has the max amount of bullets
        local maxBulletsCount = weapon:getMaxAmmo() -- caching the max ammo count possible in the gun
        if weapon:getCurrentAmmoCount() ~= maxBulletsCount then
            -- set the bullet count to max
            weapon:setCurrentAmmoCount(maxBulletsCount)
        end
    end
end

Events.OnPlayerUpdate.Add(OnPlayerUpdate)
```

Using caching we end up not having to call multiple times various functions as we do only 6 java calls instead of 13 in the first one, basically halving the function calls.

Caching within the core file

Very often some of the core objects don't change and you end up using the same object over different function calls. You can store the objects, values etc in the core of the file instead of calling it inside the function everytime. This gives a performance boost and allows you to use the same object in different functions too.

```lua
-- bad
Events.OnTick.Add(function()
    local zombieList = getCell():getZombieList()
    print(zombieList:size() .. "zombies currently loaded")
end)

-- better
local zombieList
-- Event fired when the cell loads
Events.OnPostMapLoad.Add(function(cell)
```

```
    zombieList = cell:getZombieList()
end)

Events.OnTick.Add(function()
    print(zombieList:size() .. "zombies currently loaded")
end)
```

The zombie list will always be the same object in a save, meaning you can store it on a save loading and never have to retrieve it ever again.

The less function calls, the better

Again here, we base this rule on the first and main rule to not run something if you don't need to but we go in detail on why exactly functions should be used as little as possible.

Doing a function call in Kahlua tends to be a costly operation due to function overhead. This sadly means we have to sacrifice code readability and structuration for bigger functions to greatly improve performances. This is particularly impactful in cases where functions are called a lot, such as when running code for every zombie every ticks to continue on one of the previous examples.

In this way, try to use the `math` module as little as possible, and use directly the code they run, without the need to call a function.

In continuation on this, Java functions are a costly operation to be called from the Lua and as such should be used as little as possible but of course they are the core of modding and you can't mod without them. As such, if you find alternatives which are Lua sided, it might have a point to compare the performance impact.

`math` equivalents

Here are some of the `math` lua library code to use instead of the functions.

```
-- math.max
local res = value > maxvalue and value or maxvalue

-- math.min
local res = value < minvalue and value or minvalue
```

```lua
-- math.min/max combo
local res = value > minvalue and (value < maxvalue and value or maxvalue) or minvalue

-- math.pow
local res = value ^ exponent

-- math.sqrt
local res = value ^ 0.5

-- math.floor
local res = value - value % 1

-- math.abs
local res = value < 0 and -value or value
```

You can find many more example of these and it is way better to use these than the actual functions.

Prints are the devil

If you ever get performance problems with a mod, it is actually very easy to bet on the potential problem: prints. These are used by most modders to show in the console values and actions happening in a code and are only useful for development of the mod. But this operation is extremely costly to run and will be the source of many many lags in mods. Too often do you see modders leaving these in thinking they don't matter when they are the source of most performance issues in way too many mods.

Debug mode activated or not, the performance impact is the same. Some modders add a check for debug mode to need to be activated to print in the console but here we remember the first rule: do not run it if you don't need to. Adding such checks will introduce useless code running for no reason, even if it stops prints from happening.

Prints are useless for almost everyone, users will not need it nor care about it and those who do rarely ever know what they mean. Prints are personal to the modder and shouldn't be left in uploaded code. Leaving it can also cause problems for other modders who try to develop patches for the mod that has prints as they get their console bloated with informations they do not need, hiding their own prints for their development.

Even if the prints are not done often like every in-game hours, the problem is that if every modders have prints every in-game hours, you get a massive amount of prints being called at the same time, causing lag every in-game hours. This bases itself on the principle of load balancing which is explained in the part below.

Tables

Lua tables can be key tables or array tables, even if technically they are only key tables.

Array tables

Array tables only accept integer keys (starting from 1) and the right way to create a proper array in Lua (API) is with `table.newarray()` in place of `{}`.

Proper array tables are still tables and will not break code like checking the type to be table.

Trying to access an invalid or non-existant key will throw an error instead of returning `nil` and as such some functions which take an array table in entry might break due to the use of proper arrays. If you end up having to change a proper array to a table, you are better off using tables directly.

```lua
-- bad
local myArray = {
    "entry1",
    "entry2",
    "entry3",
    "entry4",
}
print(myArray[2]) -- print second entry

-- better
local myArray = table.newarray(
    "entry1",
    "entry2",
    "entry3",
    "entry4"
)
print(myArray[2]) -- print second entry
```

```lua
-- alternative
local myArray = {
    "entry1",
    "entry2",
    "entry3",
    "entry4",
}
myArray = table.newarray(myArray) -- transform fake array into proper array
print(myArray[2]) -- print second entry
```

Proper array tables can't be saved, meaning you can't use them in mod data or send them in network commands.

`pairs` and `ipairs`

`pairs` and `ipairs` are Lua functions which are used to iterate through elements in a table (see Looping through tables). However these are slow and it is better to use alternatives when available.

Alternatives to `ipairs`

```lua
local table = {
    "Hello",
    "World",
    "!",
}

-- slower:
for i, v in ipairs(t) do
    print(v)
end

-- MUCH faster:
for i = 1, #t do
    local v = t[i]
```

```
    print(v)
end
```

Alternatives to `pairs`

Sadly the alternative way of iterating through a key table is not very pratical and as such should be used only when performance is critical. The method involves the use a table alongside a proper array.

```
-- bad
local lookup = {
    ["key1"] = "Hello",
    ["key2"] = "World",
    ["key3"] = "!",
}
for k,v in pairs(lookup) do
    print(v)
end

-- better
local lookup = {
    ["key1"] = "Hello",
    ["key2"] = "World",
    ["key3"] = "!",
}
local keys = table.newarray("key1","key2","key3")

-- iterating through the entries
for i = 1, #keys do
    local k = keys[i]
    local v = lookup[k]
    print(v)
end
```

Generating randomness

Main article: Random

The usual way to generate a random number is by using ZombRand but it generates a high quality randomness which is not necessary at all. Thankfully there is an alternative Random which is less costly:

```
-- cache newrandom
local myRandom = newrandom()

-- generate a random number
local value = myRandom:random(min,max)
```

Load balancing

A way to increase performances is to reduce how much code you run every ticks. If we continue on the example of running code for every zombies every ticks, if you have 5000 zombies loaded for the client, `OnZombieUpdate` will run 5000 times you function every ticks (in reality zombies that are not visible are not updated as often as visible ones, but still enough to matter). This can be very costly depending on what type of code you're running, even after following all the previous tips and tricks on optimization. As such you could run N zombies every ticks instead.

Queuing actions and spreading them to be ran on multiple ticks can improve greatly performances. As such you could instead of doing a function call, add the call with its parameters in a list which every N ticks will run the first operation in the list. This can be adapted in many ways, to make sure not too many operations are stacking one after the other for example, or run after different time deltas.

This system can be applied for a code which triggers thunder at a specific time in different places, by spreading them on different ticks and not create a lag spike.

Example

The code below updates one zombie per tick, making it very efficient with the default of possibly not doing the updates fast enough depending on what you are doing. You can use other methods to iterate through N zombies per ticks.

```lua
--- Snipet by Albion

-- cache the zombie list in a local variable when lauching a save
local zombieList
local function OnGameStart()
    zombieList = getPlayer():getCell():getZombieList()
end
Events.OnGameStart.Add(OnGameStart)

-- cycle 1 zombie/tick.
local zeroTick = 0
local function OnTick(tick)
    -- next zombie
    local zombieIndex = tick - zeroTick
    if zombieList:size() > zombieIndex then
        local zombie = zombieList:get(i)
        -- run code for this zombie
    else
        zeroTick = tick + 1
    end
end

Events.OnTick.Add(OnTick)
```

Benchmarking

The best way to determine what is faster to use for your code is simply benchmark it which involves counting how fast the code runs. This can be applied to small code snipets or directly to major parts of your code to see which part takes the longest to run.

To determine the time delta between two operations, you can use the functions `GameTime.getServerTime` which outputs the time in **nanoseconds**:

```lua
-- initialize getTime method
GameTime.setServerTimeShift(0) -- necessary to be able to use the following function
```

```lua
local getTime = GameTime.getServerTime -- cache the function to save some overhead

-- get the current time
local currentTime = getTime()

-- calculate the time delta since the last getTime
local timeDelta = getTime() - currentTime

-- alternative with worse precision to getting the current time
local currentTime = os.time()
```

Example

Here's an example of what benchmarking functions look like to benchmark code snipets or functions:

```lua
-- initialize getTime method
GameTime.setServerTimeShift(0) -- necessary to be able to use the following function
local getTime = GameTime.getServerTime -- cache the function to save some overhead

-- initialize variables
local totalTime = 0
local calls = 0

---Run the function and calculate the time it took
---@param fct function
function benchmark(fct,...) -- "..." is used for optional variables, these will be used in your fct
    local start = getTime() -- get start time
    fct(...) -- run your function
    local deltaTime = getTime() - start -- get time delta to run function

    totalTime = totalTime + deltaTime
    calls = calls + 1
end
```

```lua
---Print the benchmarking results in the console
function printBenchmark()
    if calls ~= 0 then
        print("Average time taken: ", totalTime / calls)
        resetBenchmark()
    else
        print("Need to benchmark at least once")
    end
end

---Reset the benchmark
function resetBenchmark()
    totalTime = 0
    calls = 0
end
```

An example usage of such functions:

```lua
-- creating an example function
local variable = 1
local function myFunction(v)
    variable = variable + v
end

-- run the benchmark 100 times for an average time
for _ = 1,100 do
  benchmark(myFunction,2)
end

printBenchmark()
```

Modeling and texturing

Due to the nature of game being isometric with a view which will tend to be far from objects, it is advised to take into in your modeling and texturing the fact most of the time the player will not have a camera close enough to the player to see details. And even in the cases where the camera is close to the model, it's usually not enough to see proper details on your textures and models.

Polycount

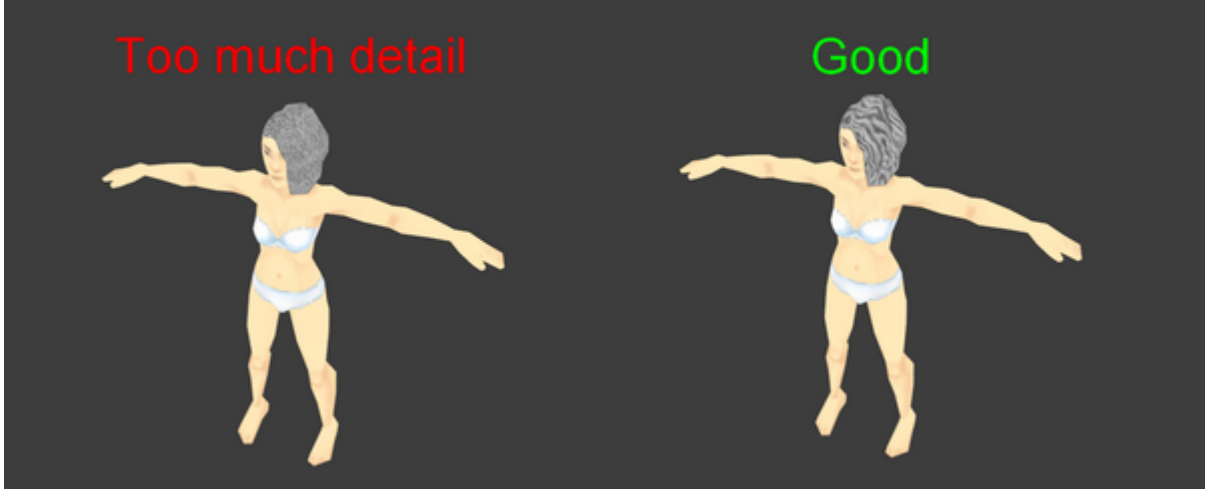
The best way to optimize your models is to keep the polycount (http://wiki.polycount.com/wiki/Polygon_Count) as low as possible, especially in the cases where it's not needed. For example, you will not have any use for a detailed earring since it will be way too small to see any kind of detail on it.

A great way to add detail to a model is to utilize the texture. You can use drawing techniques to create the illusion of topography for the player. Take this example from PALSJ (https://steamcommunity.com/id/palsj) adding detail on the model via texturing instead of making a detailed model:

Texture size limit

These rules apply to texturing too, because while you could have 4K textures, you really do not have a need for it for every game assets. See the example below from a guide to texture optimization (https://forum.game-guru.com/thread/222504), in Project Zomboid the camera will be zoomed so far back most of the times that it will be impossible to notice the texture size and it's probably best to keep it below 256x256.

The reality is that it's practically impossible to see the detail and there's even cases where too much detail will make your texture look worse. Take the example made by spongie (https://steamcommunity.com/id/LazySpongie) showcasing how the amount of detail can make a texture confusing to look at due to the amount of detail and how reducing the detail for Project Zomboid can improve the visual clarity of it.

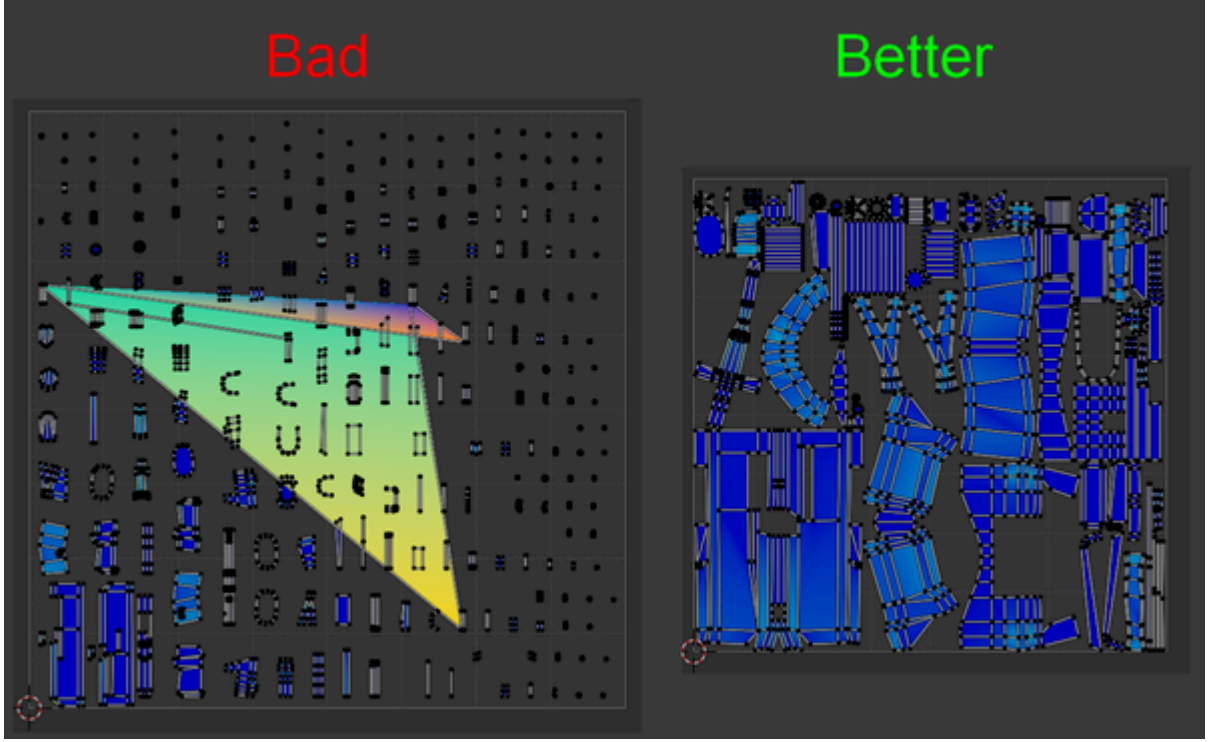


You can take the official suggestion from The Indie Stone (https://discord.com/channels/136501320340209664/869327724504842330/993893080694530128) about texture size for Project Zomboid:

> *Small request from Spiffo-on-high for those who make models for PZ mods.*
>
> *Oversized textures for weapons/clothing/vehicles etc within mods are currently becoming more of an issue with mod users - sometimes causing memory leaks and performance issues when mods stack up.*
>
> *Please try to avoid using model textures that are larger than the ones used in Vanilla PZ for equivalent items. Generally speaking vehicles are 512x512. Body and clothing textures are 256x256. Weapons are generally 128x128 - but vary due to the variation in sizes. Hats are 128x128.*
>
> *Also in the same ballpark: using tricounts higher than vanilla can also have negative effects on performance too.*
>
> *We will endeavour to help this situation along code-side, but in the meantime if people could try to keep within these limits we would be super-grateful.*

UV mapping

The best way to do UV mapping is to fit as much texture and have as little of free space on the UV map as possible. This will reduce file size and can also help you in texturing on the model. Take this example from Nik (https://steamcommunity.com/profiles/76561198046849708) on how you should optimize your UV map:



See also

You can refer to these for more detail on performances impacts. Some of the points in these are explained in the guide:

- Lua optimization guide (https://github.com/demiurgeQuantified/PZModdingGuides/blob/main/guides/Optimisation.md) by Albion

Navigation

Modding

Retrieved from "https://pzwiki.net/w/index.php?title=Mod_optimization&oldid=1359607"

Mod optimization
From PZwiki